

Assignment 09

Part 0: CS Rosters

This program relies on the following files:

- class_data.txt
- enrollment_data.txt

You've been hired by NYU's Computer Science department to create a tool that will allow professors to look up their course rosters online. Currently course registration data is stored using two different text files.

class_data.txt

Stores the course ID and the title of each course. There will always be one record in this file for every course that the CS department is currently offering. Here's what this file looks like:

```
CS0002,Introduction to Computer Programming
CS0004,Introduction to Web Design and Computer Principles
CS0060,Database Design and Implementation
CS0061,Web Development
CS0101,Introduction to Computer Science
CS0102,Data Structures
CS0201,Computer Systems Organization
CS0380,Special Topics in Computer Science
```

enrollment_data.txt

Stores the course ID, last name and first name of each student enrolled in the course. Note that there are multiple records per course (one per student enrolled). For example, here's the beginning of this file showing six students enrolled in CS0002:

```
CS0002,Hiner,Judith
CS0002,Mcmahon,Ludivina
CS0002,Trusty,Beatrice
CS0002,Brinn,Jacquelyn
CS0002,Hintzen,Floria
CS0002,Amyx,Randolph
```

Your task is to write a program that asks the user for a course ID. The program will then determine if the course ID is valid or not -- if it is not, the program can safely end. If it is valid, the program

should report the full title of the course, how many students are enrolled in the course and the names of each student enrolled, in alphabetical order by last name, and in the format First Last.

You cannot hard code your program to only work with the values contained within these files.

When we test your work, we will be using completely different data files that are organized in the same way as these sample files. Your program should work flawlessly using this new data.

Here are a few sample runnings of how this program should operate (user input is underlined and runs are separated by -----):

```
NYU Computer Science Registration System
```

```
Enter a course ID (i.e. CS0002, CS0004): peanut butter
```

```
Cannot find this course
```

```
-----
```

```
NYU Computer Science Registration System
```

```
Enter a course ID (i.e. CS0002, CS0004): CS0002
```

```
The title of this class is: Introduction to Computer Programming
```

```
The course has 6 students enrolled
```

- * Randolph Amyx
- * Jacquelyn Brinn
- * Judith Hiner
- * Floria Hintzen
- * Ludivina McMahon
- * Beatrice Trusty

```
-----
```

```
NYU Computer Science Registration System
```

```
Enter a course ID (i.e. CS0002, CS0004): CS0004
```

```
The title of this class is: Introduction to Web Design and Computer Principles
```

```
The course has 7 students enrolled
```

- * Jerrica Alfrey
- * Edgar Cales
- * Glayds Conn
- * Judith Hiner
- * Refugio Lukens
- * Saran Schneiderman
- * Tilda Woodman

NYU Computer Science Registration System

Enter a course ID (i.e. CS0002, CS0004): CS0201

The title of this class is: Computer Systems Organization

The course has 0 students enrolled

This program should be named as follows: LastNameFirstName_assign9_part0.py

Problem 0 Points (27):

- Course ID is validated (test a random string and ensure “Cannot find this course” is printed) (3 points)
- Names are printed in alphabetical order (3 points)
- Graders will be using a new version of class and enrollment data, formatted the same as the files you are working with. They will test 3 courses worth 7 each. For each example:
 - Correct title is printed (2 points)
 - Correct number of students are printed (2 points)
 - Correct student names are printed (3 points)

Part 1: Email Prototype

Note that the programs you will be writing build on one another. You will want to attempt these parts in order. You can also feel free to save all of your work in one big file (call it "LastNameFirstName_assign9_part1.py")

You have just been hired by a tech startup that is planning on releasing a new e-mail service to its clients. They have asked you to create a prototype of this service using your Python skills. The final version of this prototype should do the following:

- Allow the user to **register for a new account**. When they register, they will supply a **username and password**, which will need to be validated. The username will also need to be unique (i.e. only one person can have a given username). If the user registers successfully their information will need to be **stored in a permanent storage mechanism** (in this case, a **file**)
- The user should also be able to **log into their account**. This part of the system will ask the user for a **username and password and match** it up against the **one the company has** on file for them. Note that the company plans on having millions of clients, so this information must be cross-referenced against a file and cannot be 'hard-coded'.
- The user should be able to **send messages to any other user on the system**. To send a message the system will ask the user for the **name** of a recipient and a **message to send** (one line of text). If the recipient is registered the message will be delivered through a file.
- The user should be able to **read messages that have been sent** to them (and only them)
- The user should be able to **delete all of the messages that have been sent to them** (and only them)

In order to do this the software engineers at the company have outlined a series of functions that will need to be built to create the prototype. Your job is to build these functions and test them as

you go - don't skip ahead! When all of the necessary functions are built you will use them to construct the prototype e-mail system.

Part 1a

Begin by writing functions that conform to the following IPO specifications. Tester code is listed below each function.

```
# function:    valid_username
# input:       a username (string)
# processing:  determines if the username supplied is valid.  for the purpose
#             of this program a valid username is defined as follows:
#             (1) must be 5 characters or longer
#             (2) must be alphanumeric (only letters or numbers)
#             (3) the first character cannot be a number
# output:      boolean (True if valid, False if invalid)

# TESTER CODE
print( valid_username('abc123') )    # True
print( valid_username('abcde') )     # True
print( valid_username('abc') )       # False
print( valid_username('@#$%^') )    # False
print( valid_username('1abcde') )    # False
print( valid_username('') )          # False

# function:    valid_password
# input:       a password (string)
# processing:  determines if the password supplied is valid.  for the purpose
#             of this program a valid password is defined as follows:
#             (1) must be 5 characters or longer
#             (2) must be alphanumeric (only letters or numbers)
#             (3) must contain at least one lowercase letter
#             (4) must contain at least one uppercase letter
#             (5) must contain at least one number
# output:      boolean (True if valid, False if invalid)
```

```
# TESTER CODE
print( valid_password('Abc123')      )    # True
print( valid_password('Abc123xyz')    )    # True
print( valid_password('Ab12')         )    # False
print( valid_password('abc123')       )    # False
print( valid_password('123456')       )    # False
print( valid_password('Abc123#')      )    # False
print( valid_password('')             )    # False
```

Part 1b

Now that you can validate usernames and passwords it's time to write some code that will **register new user accounts**. In order to do this, you will need to create a file that will contain the usernames and passwords for all users of the system. This file, called **user_info.txt**, is set up with five demo accounts and is organized as follows:

```
pikachu,Abc123
charmander,Xyz123
squirtle,SquirtleSquad99
Pidgey2020,Pqr123
fearow,Pidgey2020
```

- The file is organized into multiple lines
- Each line represents a single user
- On each line there are two values, separated by commas. The first value (before the comma) is the username and the second value (after the comma) is the password for that user.

The first thing you will want to do is **download this file and save** it in the same folder as your source code file for this program. Once this file is in place you can begin writing the following two functions:

```
# function:    username_exists
# input:       a username (string)
# processing:  determines if the username exists in the file 'user_info.txt'
# output:      boolean (True if found, False if not found)
```

```

# TESTER CODE
print( username_exists('pikachu') ) # True
print( username_exists('charmander') ) # True
print( username_exists('squirtle') ) # True
print( username_exists('Pidghey2020') ) # True
print( username_exists('SquirtleSquad99') ) # False
print( username_exists('eevee') ) # False
print( username_exists('bobcat') ) # False
print( username_exists('') ) # False

# function: check_password
# input: a username (string) and a password (string)
# processing: determines if the username / password combination
# supplied matches one of the user accounts represented
# in the 'user_info.txt' file
# output: boolean (True if valid, False if invalid)

# TESTER CODE
print( check_password('pikachu', 'Abc123') ) # True
print( check_password('squirtle', 'SquirtleSquad99') ) # True
print( check_password('fearow', 'Pqr123') ) # False
print( check_password('foobar', 'Hello123') ) # False
print( check_password('', '') ) # False

```

Part 1c

For this part you will be writing a function that will **create new users for the system**. This function should be designed as follows:

```

# function: add_user
# input: a username (string) and a password (string)
# processing: if the user being supplied is not already in the
# 'user_info.txt' file they should be added, along with
# their password.
# output: boolean (True if added successfully, False if not)

```

```
# TESTER CODE
add_user('foobar', 'abcABC123')
add_user('barfoo', 'xyz123ABC')
add_user('foobar', 'aTest123') # this should fail

# OUTPUT – check 'user_info.txt' to ensure that that two new accounts have be
en created
```

Part 1d

Next you will need to write a function to allow users to send messages to one another. Messages will be stored in a folder named 'messages' which you will need to create. Your file structure should look as follows:

assignment09/	(folder)
LastNameFirstName_assign9_part1.py	(main program)
user_info.txt	(user database)
messages/	(folder)

After you have manually created this 'messages' folder you be able to start sending messages from one user to another. When a user is registered with the system we will create a new file in this 'messages' folder for that user - the name of this file will be their username with the file extension '.txt' - for example, if I register an account called 'snorlax' a file called 'snorlax.txt' should be created inside of the 'messages' folder for this user. All messages that will be sent to this user will be stored in this file.

Note that you can easily create, edit and read files in sub-folders by using this syntax:

```
file = open('messages/somefile.txt', 'w')
# opens a new file named 'somefile.txt' for writing inside of the 'messages'
folder
```

Here's the function IPO for the new function that can be used to send messages:

```
# function:    send_message
# input:       a sender (string), a recipient (string) and a message (string)
# processing:  writes a new line into the specific messages file for the given
#              users with the following information:
#              sender|date_and_time|message\n
```

```
#         for example,if you call the function using following arguments:
#         send_message('craig', 'pikachu', 'Hello there! nice to see you!')
#         the file 'messages/pikachu.txt' should gain an additional
#         line of data that looks like the following:
#         craig|11/14/2020 12:30:05|Hello there! nice to see you!\n
#         note that you can generate the time by doing the following:
#         import datetime
#         d = datetime.datetime.now()
#         month = d.month
#         day = d.day
#         year = d.year
#         ... etc. for hour, minute and second
#
#         keep in mind that you may need to 'append'
#         to the correct messages file since a user can receive
#         an unlimited number of messages. you may also need to create a
#         new message file if one does not exist for a user.
# output:  nothing
```

```
# TESTER CODE
```

```
send_message('pikachu', 'charmander', 'Hey there!')
send_message('charmander', 'pikachu', 'Good to see you!')
send_message('pikachu', 'charmander', 'You too, ttyl')
```

```
# OUTPUT – two new messages files should be created – 'pikachu.txt' and 'charmander.txt'
```

```
# each should have the following content (dates will be different, though)
```

```
#
```

```
# pikachu.txt
```

```
# charmander|11/14/2020 13:37:15|Good to see you!
```

```
#
```

```
# charmander.txt
```

```
# pikachu|11/14/2020 13:37:15|Hey there!
```



```
# pikachu|11/14/2020 13:37:15|You too, ttyl
```

Once this is working as expected you should update your 'add_user' function so that every time a new user is added a default message is sent to that user welcoming them to the system. This message should be sent from the user account 'admin' and should contain the text 'Welcome to your account!'

Part 1e

Finally, you will be writing two additional functions to complete the tools needed to write this program. The **first function** you will be writing will **print out all messages for a specific user**. Here's the IPO for this function:

```
# function:    print_messages
# input:       a username (string)
# processing:  prints all messages sent to the username in question.  assume y
ou have this file named 'pikachu.txt':
#
#             charmander|11/14/2020 13:37:15|Hey there!
#             charmander|11/14/2020 13:37:15|You too, ttyl
#
#             this function should generate the following output:
#
#             Message #1 received from charmander
#             Time: 11/14/2020 13:37:15
#             Hey there!
#
#             Message #2 received from charmander
#             Time: 11/14/2020 13:37:15
#             You too, ttyl
# output:     no return value (simply prints the messages)
```

Once you can print messages successfully the last function you will need to **write will be a function to delete all messages for a given user**. The IPO for this function is as follows:

```
# function:    delete_messages
# input:       a username (string)
# processing:  erases all data in the messages file for this user
# output:     no return value
```

Part 1f

Now you are ready to write the main program for this project! Here's what your program should do:

- Present the user with a menu of choices - login, register or quit
- If they choose to quit you should immediately end the program.
- If they choose to register you should prompt them for a username and a password. Ensure that the username and password are valid using your functions. Then ensure that the username has not already been taken. If all of these items are OK you can go ahead and register the user for an account and return them back to the main menu.
- If they choose to login you should prompt them for a username and a password. If the username and password is correct they should be logged in. They then should receive a new menu of options (hint - user a nested 'while' loop for this):
 - Present the user with the options to read messages, send messages, delete messages or logout
 - If they choose to logout they should return to the main menu
 - If they choose to read messages you should display their messages
 - If they choose to delete messages you should delete all their messages
 - If they choose to send a message you should prompt them for a recipient username and a message. If the recipient is unknown you should reject the message. If they are known, you should send a message to that user using your function.

Here's a sample running of the program. This program assumes that you have an empty "user_info.txt" file (no users registered) and an empty 'messages' folder (folder exists, but no files are stored inside of it). User input is underlined:

```
(l)ogin, (r)egister or (q)uit: r

Register for an account
Username (case sensitive): snorlax
Password (case sensitive): XYZ!@#$
Password is invalid, registration cancelled

(l)ogin, (r)egister or (q)uit: r

Register for an account
Username (case sensitive): snorlax
Password (case sensitive): Abc123
Registration successful!

(l)ogin, (r)egister or (q)uit: r
```

Register for an account

Username (case sensitive): caterpie

Password (case sensitive): Xyz123

Registration successful!

(l)ogin, (r)egister or (q)uit: r

Register for an account

Username (case sensitive): snorlax

Password (case sensitive): Hacker1

Duplicate username, registration cancelled

(l)ogin, (r)egister or (q)uit: l

Log In

Username (case sensitive): snorlax

Password (case sensitive): Abc123

You have been logged in successfully as snorlax

(r)ead messages, (s)end a message, (d)elele messages or (l)ogout: r

Message #1 received from admin

Time: 11/14/2020 13:56:12

Welcome to your account!

You have been logged in successfully as snorlax

(r)ead messages, (s)end a message, (d)elele messages or (l)ogout: s

Username of recipient: caterpie

Type your message: Hey there!

Message sent!

You have been logged in successfully as snorlax

(r)ead messages, (s)end a message, (d)elele messages or (l)ogout: s

Username of recipient: caterpie

Type your message: How are you??

Message sent!

You have been logged in successfully as snorlax

(r)ead messages, (s)end a message, (d)elele messages or (l)ogout: s

Username of recipient: pikachu

Unknown recipient

You have been logged in successfully as snorlax

(r)ead messages, (s)end a message, (d)elele messages or (l)ogout: l

Logging out as username snorlax

(l)ogin, (r)egister or (q)uit: l

Log In

Username (case sensitive): caterpie

Password (case sensitive): Xyz123

You have been logged in successfully as caterpie

(r)ead messages, (s)end a message, (d)elele messages or (l)ogout: r

Message #1 received from admin

Time: 11/14/2020 13:56:30

Welcome to your account!

Message #2 received from snorlax

Time: 11/14/2020 13:57:00

Hey there!

Message #3 received from snorlax

Time: 11/14/2020 13:57:11

How are you??

You have been logged in successfully as caterpie

(r)ead messages, (s)end a message, (d)elele messages or (l)ogout: s

Username of recipient: snorlax

Type your message: Great! How are you?

Message sent!

You have been logged in successfully as caterpie

(r)ead messages, (s)end a message, (d)eleete messages or (l)ogout: d

Your messages have been deleted

You have been logged in successfully as caterpie

(r)ead messages, (s)end a message, (d)eleete messages or (l)ogout: r

No messages in your inbox

You have been logged in successfully as caterpie

(r)ead messages, (s)end a message, (d)eleete messages or (l)ogout: l

Logging out as username caterpie

(l)ogin, (r)egister or (q)uit: l

Log In

Username (case sensitive): snorlax

Password (case sensitive): Abc123

You have been logged in successfully as snorlax

(r)ead messages, (s)end a message, (d)eleete messages or (l)ogout: r

Message #1 received from admin

Time: 11/14/2020 13:56:12

Welcome to your account!

Message #2 received from caterpie

Time: 11/14/2020 13:57:39

Great! How are you?

You have been logged in successfully as snorlax

(r)ead messages, (s)end a message, (d)eleete messages or (l)ogout: l

Logging out as username snorlax

(l)ogin, (r)egister or (q)uit: q

Goodbye!

Important note: your program should be able to be "re-run" after the sample code above and pick up where it left off due to the fact that relevant data is being stored in a series of permanent files on your computer. For example, the following code was run days after the original output above - note how the system is able to "pick up" where it left off after the first run.

(l)ogin, (r)egister or (q)uit: r

Register for an account

Username (case sensitive): snorlax

Password (case sensitive): AnotherHack1

Duplicate username, registration cancelled

(l)ogin, (r)egister or (q)uit: l

Log In

Username (case sensitive): caterpie

Password (case sensitive): XYZ123

You have been logged in successfully as caterpie

(r)ead messages, (s)end a message, (d)eleete messages or (l)ogout: r

No messages in your inbox

You have been logged in successfully as caterpie

(r)ead messages, (s)end a message, (d)eleete messages or (l)ogout: s

Username of recipient: snorlax

Type your message: Are you still there?

Message sent!

You have been logged in successfully as caterpie

(r)ead messages, (s)end a message, (d)eleete messages or (l)ogout: l

Logging out as username caterpie

(l)ogin, (r)egister or (q)uit: l

Log In

Username (case sensitive): snorlax

Password (case sensitive): Abc123

You have been logged in successfully as snorlax

(r)ead messages, (s)end a message, (d)elele messages or (l)ogout: r

Message #1 received from admin

Time: 11/14/2020 13:56:12

Welcome to your account!

Message #2 received from caterpie

Time: 11/14/2020 13:57:39

Great! How are you?

Message #3 received from caterpie

Time: 11/16/2020 15:59:10

Are you still there?

You have been logged in successfully as snorlax

(r)ead messages, (s)end a message, (d)elele messages or (l)ogout: l

Logging out as username snorlax

(l)ogin, (r)egister or (q)uit: q

Goodbye!

Note on submitting your work: please submit a 'ZIP' file of your work, which includes your main program, the 'user_info.txt' file and your 'messages' folder. On a Mac you can create a 'ZIP' file by highlighting these three files and control-clicking on them - then select 'Compress Items' - on a PC you can highlight these files and right-click, then select 'Send To' and then 'Compressed (zipped) folder'. Please delete all files from the 'messages' folder before uploading your assignment. Also

please delete all contents from the 'user_info.txt' file so that no users are registered. You should submit a 'user_info.txt' file, but it should be empty.

Problem 1 Points (73):

3 points for submitting a ZIP file with proper structure (your program, user_info.txt, and messages folder). 5 points for each of the tests below. Note that the program should work across multiple runs and proper files should be created when applicable for this to work.

- When 'q' is entered, the program ends and "Goodbye!" is printed
- When an invalid username is entered, registration is canceled
- When a valid username and password are entered, registration is successful and saved
- When a duplicate username is entered for registration, registration is canceled
- Incorrect username results in invalid log-in
- Incorrect password results in invalid log-in
- Correct username and password combination results in log-in; "You have been logged in successfully as [username]" is printed
- Once logged in:
 - When messages are read, a welcome message from the admin with a timestamp is printed
 - When trying to send a message to a username that does not exist, 'unknown recipient' is printed
 - [You will need at least two users to test this] When trying to send a message to a user that does exist, 'Message sent!' is printed
 - When you log-in to the other account and read messages, this message along with a timestamp and the username of who sent the message are printed
 - Messages can be deleted
 - Once messages are deleted, when messages are read, 'No messages in your inbox' is printed
 - Able to log-out of account